

Лабораторная работа №7

Автор: Ежова Екатерина

Группа: 6203-010302D

Задание 1

Чтобы все объекты типа `TabulatedFunction` можно было использовать в качестве объекта-агрегата в «улучшенном цикле `for`», в интерфейсе `TabulatedFunction` я добавила родительский тип `Iterable<FunctionPoint>`. В каждом из классов (`ArrayTabulatedFunction` и `LinkedListTabulatedFunction`), реализующих этот интерфейс, я добавила метод, возвращающий объект итератора `iterator()`. Классы итераторов я сделала анонимными. В них я переопределила 3 метода: `hasNext()`, `next()` и `remove()`. Первый метод проверяет, существует ли следующий элемент, второй – возвращает его, третий метод выбрасывает исключение `UnsupportedOperationException` при вызове.

```
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int index = 0; // 2 usages
        @Override
        public boolean hasNext() {
            return index < arrayLen;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Следующего элемента не существует");
            }
            return arrayPoints[index++];
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Операции удаления нет");
        }
    };
}
```

Вывод в методе `main()`:

```
Задание 1
fun1:
(1.0; 3.0)
(12.0; 36.0)
(23.0; 69.0)
(34.0; 102.0)
(45.0; 135.0)
(56.0; 168.0)
(67.0; 201.0)
(78.0; 234.0)
(89.0; 267.0)
(100.0; 300.0)

fun2:
(1.0; 1.0)
(2.0; 2.0)
(3.0; 3.0)
(4.0; 5.0)
(5.0; 9.0)
(6.0; 6.0)
(7.0; 32.0)

System.out.println("Задание 1\fun1:");
TabulatedFunction fun1 = new ArrayTabulatedFunction( leftX: 1, rightX: 100, pointsCount: 10);
for (int i = 0; i < fun1.getPointsCount(); i++) {
    fun1.setPointY(i, y: 3 * fun1.getPointX(i));
}

double[] val = {1, 2, 3, 5, 9, 6, 32};
TabulatedFunction fun2 = new LinkedListTabulatedFunction( leftX: 1, rightX: 7, val);
for (FunctionPoint p : fun1) {
    System.out.println(p);
}

System.out.println("\nfun2:");
for (FunctionPoint p : fun2) {
    System.out.println(p);
}
```

Задание 2

В пакете functions я описала интерфейс TabulatedFunctionFactory, содержащий три перегруженных метода createTabulatedFunction(). Первый метод принимает в качестве аргумента массив типа FunctionPoint, второй – левую и правую границы табулирования и количество точек для табуляции, третий метод вместо количества точек принимает массив значений функции на промежутке. В классах ArrayTabulatedFunction и LinkedListTabulatedFunction я создала вложенные классы ArrayTabulatedFunctionFactory и LinkedListTabulatedFunctionFactory, которые реализуют интерфейс TabulatedFunctionFactory и с помощью конструктора создают объекты того класса, в котором они находятся.

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory { //вложенный класс фабрики

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] values) { //переопределяем методы и возвращаем
        return new ArrayTabulatedFunction(values); // с помощью конструктора создаем новый объект
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }
}
```

В классе TabulatedFunctions я объявила приватное статическое поле factory типа TabulatedFunctionFactory и проинициализировала его объектом класса ArrayTabulatedFunctionFactory. Также я объявила метод setTabulatedFunctionFactory(), позволяющий заменить объект фабрики. В этом классе я объявила 3 перегруженных метода createTabulatedFunction(), которые возвращают объекты табулированных функций, созданные с помощью текущей фабрики. В остальных методах класса (tabulate(), inputTabulatedFunction() и readTabulatedFunction()), где требуется создание объектов табулированных функций, явное создание объектов с помощью конструкторов я заменила на вызов метода createTabulatedFunction().

Вывод в методе main():

```
Задание 2
class functions.ArrayTabulatedFunction
class functions.LinkedListTabulatedFunction
class functions.ArrayTabulatedFunction
```

Задание 3

В этом задании я перегрузила 3 метода `createTabulatedFunction()`, добавив в каждом методе еще один параметр – ссылку типа `Class` на описание класса, объект которого требуется создать. В эти методы можно передать только ссылки на классы, реализующие интерфейс `TabulatedFunction`. Далее с помощью метода `getConstructor()` находим конструктор по типу передаваемых параметров и с помощью метода `newInstance()` создаем и возвращаем объект табулированной функции. В блоке `try` отлавливаем все возможные исключения. Также я перегрузила методы `(tabulate(), inputTabulatedFunction() и readTabulatedFunction())`, создающие объекты табулированных функций, сделав так, чтобы они принимали ссылку типа `Class` на описание класса, объект которого требуется создать.

```
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> cl, FunctionPoint[] values) {
    try {
        Constructor<? extends TabulatedFunction> constructor = cl.getConstructor(FunctionPoint[].class); //находим конструктор
        return constructor.newInstance((Object) values); //вызываем конструктор
    } catch (NoSuchMethodException e) {
        throw new IllegalArgumentException("В классе нет такого конструктора", e);
    } catch (InstantiationException e) {
        throw new IllegalArgumentException("Нельзя создать объект абстрактного класса", e);
    } catch (IllegalAccessException e) {
        throw new IllegalArgumentException("Нет доступа к конструктору", e);
    } catch (InvocationTargetException e) {
        throw new IllegalArgumentException("Конструктор выбрасывает исключение", e);
    }
}
```

Вывод в методе `main()`:

```
Задание 3
class functions.ArrayTabulatedFunction
{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}
class functions.ArrayTabulatedFunction
{(0.0; 0.0), (10.0; 10.0)}
class functions.LinkedListTabulatedFunction
{(0.0; 0.0), (10.0; 10.0)}
class functions.LinkedListTabulatedFunction
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172; 0.9510565162951535), (1.5707963267948966; 1.0)}
class functions.LinkedListTabulatedFunction
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172; 0.9510565162951535), (1.5707963267948966; 1.0)}
```